Branch and Bound and Approximation methods

How to deal with NP-hard problems?

- Some problems cannot be solve in polynomial time (NP-hard problems), but sometimes we really need to find a (good) solution (TSP, Knapsack, ...)
- To have an exact solution, we have to use techniques that may be exponential in the worst case
- The naive brute-force-search approach consists in enumerating all the possible solutions and finally output the best one.
- To use such an approach, one must at least take care of some important points:
  - we need a procedure able to enumerate all the solutions, without proposing the same one many times (more than once)
  - we need a procedure able to build quickly a new solution from a previous one
  - we wish that the enumeration would be factorizable, which means that some solutions share a common part that does not have to be recomputed at each step.
- The other way is to look for approximate solutions thanks to some heuristics or particular settings

Enumeration of the solutions

To explore the solution space

- we need a method for building a solution in order to explore the solution space by following a tree structure: when a solution is not optimal, we come back (**backtrack**) to the previous choice point and take another branch of the tree $\rightarrow$ we explore a new solution set.
- Or, we could define a graph on the whole set of solutions and look for an Hamiltonian path in this graph!

Problem: Coloring the nodes of a graph

Input: a graph $G = (V, E)$ and an integer k

Question: Is there a coloring of G using at most k colors such that for two vertices linked by an edge do not have the same color: $\forall v \in V$, $1 \leq color(v) \leq k$ and $\nexists (v, v') \in E$ s.t. $color(v) = color(v')$?

A solution is represented by a list of colors, one for each node in the graph, ie a string of size n (assuming $|V| = n$) on the alphabet $\{1, \ldots, k\}$

An example for the graph coloring problem

A way to enumerate the solutions is given by the following (simple) algo:

Input: $G = (V, E)$, k: number of colors to use, *color*: (partial) list of the colors of the nodes, i is the index of the first node of V not colored.
Output: `true` or `false`

```
if i > |V| then
    return true
else
    foreach c from 1 to k do
        color(i) ← c;
        ok ←true;
        foreach (i, j) ∈ E s.t. j < i do
            if color(i) = color(j) then
                ok ← false; break out from the loop;
            if ok then
                if TryTestColoring(G, k, color, i + 1) is true then
                    return true;
    return false;
```

Algorithm 3: *TryTestColoring*$(Gg, k, color, i)$ algorithm

Using a graph containing all solutions

Using an Hamiltonian path

$V = \{C_i$ string of size n over $\{1 \dots k\}\}$,
$E = \{(C_i, C_j)$ s.t. C_i and C_j give the same color to all the nodes of V except $1\}$
Question: when using an Hamiltonian path how to generate a new solution from another one in $O(1)$ without considering twice the same permutation?

Using these approaches is not possible for some problems

- *Binpacking:* there is an infinite number of placement for each object (w.r.t. to the decimal precision)
- *TSP:* when $n > 100$ "codable" but hardly tractable

The Branch and bound approach

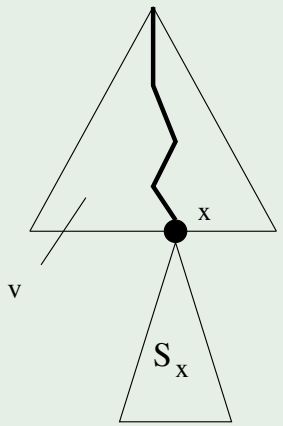
In the context of a minimization problem

- We explore the solution using a tree structure, but we cut some branches of the tree to reduce the search space of the solutions.
- We define $g(sol)$ the function we want to optimize (we assume we are looking for a minimum over g). Its definition depends on the problem considered. For example in the TSP problem it corresponds to the length of the cycle defined by the solution.

The Branch and bound approach

Principle

- we define a function (algorithm) associating quickly (polynomial time) a value $f(x)$ to each node x in the search tree. It gives the best possible value for any solution belonging to the subtree rooted at x. **For the root x we look for a lower bound for the solutions in the subtree rooted by x:** $\forall y \in S_x, f(x) < f(y)$.



- assume that at a given step, the best solution found has a value v, then in the remaining part of the tree it is not necessary to check subtrees rooted in x when $v < f(x)$.

The Branch and bound approach

In practice, a greedy heuristic (fast) is used to find a first v close to the optimal in order to cut quickly a lot of branches in the search tree.

The quality of this approach depends on how well f can approximate g.

Note: f is a heuristic, because if we would have known how to find $g(x)$ in polynomial time, we could solve the problem in polynomial time!

Example with TSP

We use a search tree: a path in the tree represents a sequence of nodes to visit, during a top-down search we obtain partial solutions and we add new nodes during the search.

We define a partial solution as: $(k, [i_1, \ldots, i_k], w)$ where w is the cost associated to partial solution.

$S_{partial} = (k, [i_1, \ldots, i_k], w)$;
$best = (n, [i_{1_b}, \ldots, i_{n_b}], w_b)$;
begin
  if $k = n$ **then**
    $cost \leftarrow w + A[i_k, i_1]$;
    if $cost < w_b$ **then**
      $best = (k, [i_1, \ldots, i_k], cost)$;
  else
    foreach $j \notin [i_1, \ldots, i_k]$ **do**
      $N_s = (k+1, [i_1, \ldots, i_k, j], w + A[i_k, j])$;
      $TSP - naive(N_s, best)$;
end

Algorithm 4: $TSP - naive(S_{partial}, best)$

The Branch and bound solution !

$S_{partial} = (k, [i_1, \ldots, i_k], w)$;
$best = (n, [i_{1_b}, \ldots, i_{n_b}], w_b)$;
begin
  if $k = n$ **then**
    $cost \leftarrow w + A[i_k, i_1]$;
    if $cost < w_b$ **then**
      $best = (k, [i_1, \ldots, i_k], cost)$;
  else
    foreach $j \notin [i_1, \ldots, i_k]$ **do**
      $cost \leftarrow w + A[i_k, j]$;
      if $cost < w_b$ **then**
        $N_s = (k+1, [i_1, \ldots, i_k, j], w + A[i_k, j]$;
        $TSP - branch - naive(N_s, best)$;
end

Algorithm 5: $TSP - branch - naive(S_{partial}, best)$

Another problem: Task assignment

Problem

We consider the following task assignment problem: we have n agents and n tasks and the problem is to associate a task to each agent in order to minimize the global cost. Indeed, each agent performs differently on the different tasks, and we assume to have a table c indicating for each agent the cost associated to each task. An example of a cost table is provided in Table 1.

Agent\Task	1	2	3	4
1	5	4	13	8
2	6	1	7	11
3	8	6	8	7
4	9	4	6	11

Table 3: Example of a cost table. The value 6 at position $c[2,1]$ indicates that for agent number 2 the cost of achieving task 1 is 6.

Question 1: How to encode a solution?

We can represent a solution as a vector where the first attribute is the number of the task affected to the 1st agent, the second attribute is the task affected to the second agent, and so on.

Given n tasks and n agents, the solutions are the set of permutations of n elements.

Given 4 tasks/agents, a partial solution can be represented as $(2, 3, ?, ?)$ meaning that the first agent receives task 2, the second agent task 3, agents 3 and 4 do not have any task at the moment.

Q2: Evaluation function?

Now, we would like to define an evaluation function f.
We want this function f to be decomposed in two terms: g^* and h, where

- g^* corresponds to the real cost of the partial solution found so far
- and h is a lower bound on the expected cost for the remaining tasks that have to selected in the solution.

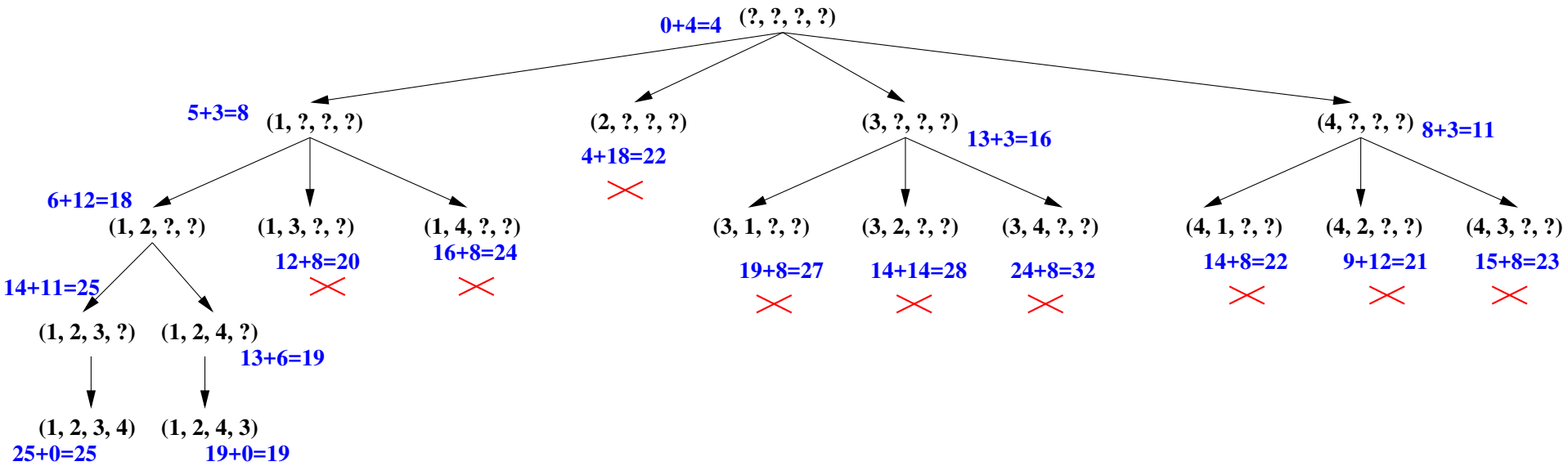
Q2: cost evaluation

Imagine that we have currently affected a task for k agents out of n, for a corresponding solution vector $\mathbf{v}$, we can define the following quantities:

- $g^*(\mathbf{v}) = \sum_{i=1}^{k} c[i, \mathbf{v}[i]]$
  the sum of the costs of the tasks affected to the first k agents
- $$h(\mathbf{v}) = (n-k) \min_{\substack{k<i\leq n \\ 1\leq j\leq n \\ j\neq \mathbf{v}[l] \text{ for } 1\leq l\leq k}} c[i,j]$$

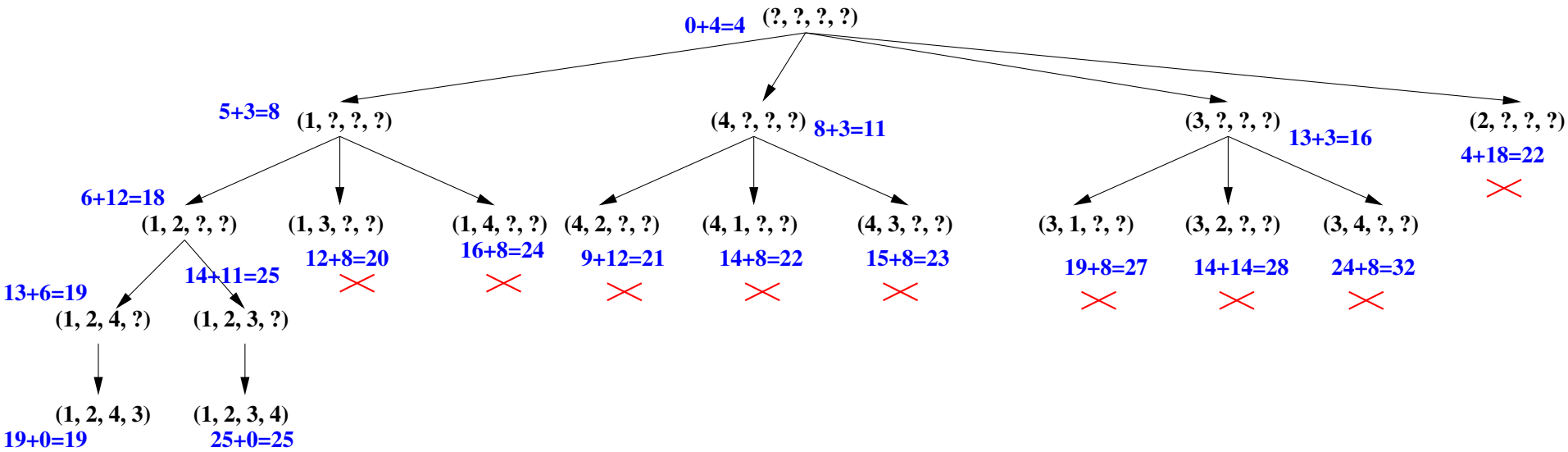
  $(n-k)$ times the minimum possible values among remaining tasks and agents

Q2: Search Tree



Each node defines a (partial) solution written in black. We associate the quantity $g^* + h = f$ in blue next (or below) to each node (first value before $=$ is g^*, second is h). The red crosses indicate when the search can be cut.

Q2: Search Tree - Best Approach



We should check the most promising path on the left side of the tree and the worst one on the right to promote early cuts in the tree. So for each node, we should consider the children with respect to increasing values given by f.

Remarks

- Quality of the approximation function is key for efficiency (Task assignment cost function can be improved)
- Another idea for TSP: adding edges instead of nodes $\rightarrow$ see exercise in class
- Another strategy for TSP: find an approximation in an Euclidean space $\rightarrow$ see exercise in class
- Heuristics depend generally on the problem considered
- Other directions: Randomized approaches, genetic algorithms ...

Branch and Bound: Summary and take home message

- Branch and Bound methods are used to solve complex problems, generally NP-hard problems.
- After a characterization of a search space of the solutions, the strategy consists in exploring this search space in a neat and efficient way by trying to "cut" some branches that cannot lead to optimal solutions.
- Other strategies include the use of heuristics, strong hypothesis on the problem, optimization methods or randomized approaches.
- The same strategy can be used to solve some NP-hard problems that are not necessarily optimization problems.